

# Informe Técnico

## “Ruta óptima entre ciudades Grafos ponderados”

Integrantes: Ankatu Alcaman,

- Fabian Neira,
- Francisco Martinez,
- Jenniffer Urrutia.

Asignatura: Matemáticas Discretas

Profesor: Elliot Jamil Mardones Arias

Fecha de entrega: 01/07/2026

## 1. Introducción

El presente informe detalla el diseño e implementación de una solución computacional basada en la teoría de grafos para resolver el problema del camino mínimo. El objetivo principal es modelar una red interurbana y calcular la ruta más eficiente entre una ciudad de origen y una de destino. Para este proyecto, se seleccionó un conjunto de 15 ciudades pertenecientes a Colombia, estableciendo un mínimo de 20 conexiones reales entre ellas. El criterio de ponderación (peso) asignado a cada arista corresponde a la distancia estimada en kilómetros entre las ciudades conectadas, manteniendo este parámetro de forma consistente para evaluar el costo de los recorridos.

## 2. Modelamiento Matemático

El sistema interurbano se representó mediante un grafo ponderado no dirigido, definido formalmente como  $G=(V,E,w)$

- **Vértices (V):** Corresponde al conjunto finito de 15 ciudades colombianas seleccionadas.

$V = \{\text{Bogotá, Medellín, Cali, Barranquilla, Cartagena, Bucaramanga, Cúcuta, Pereira, Manizales, Santa Marta, Villavicencio, Ibagué, Pasto, Montería, Sincelejo}\}$

- **Aristas (E):** Representa el conjunto de 22 conexiones viales reales habilitadas entre los nodos del grafo. Definido explícitamente mediante extensión como:

$E = \{(\text{Bogotá, Medellín}), (\text{Bogotá, Ibagué}), (\text{Bogotá, Villavicencio}), (\text{Medellín, Cali}), (\text{Medellín, Manizales}), (\text{Medellín, Bucaramanga}), (\text{Medellín, Cartagena}), (\text{Cali, Ibagué}), (\text{Cali, Pasto}), (\text{Barranquilla, Cartagena}), (\text{Barranquilla, Santa Marta}), (\text{Cartagena, Sincelejo}), (\text{Sincelejo, Montería}), (\text{Bucaramanga, Cúcuta}), (\text{Bucaramanga, Bogotá}), (\text{Cúcuta, Manizales}), (\text{Pereira, Manizales}), (\text{Pereira, Ibagué}), (\text{Pereira, Cali}), (\text{Santa Marta, Sincelejo}), (\text{Villavicencio, Ibagué}), (\text{Montería, Medellín})\}$

- **Función de peso ( $w$ ):** Es una función  $w: E \rightarrow \mathbb{R}^+$  (reales positivos) que asocia a cada arista  $(u,v)$  perteneciente a  $E$  un valor correspondiente a la distancia en kilómetros. Por ejemplo,  $w(\text{Bogotá, Medellín}) = 415$ .

### Propiedades Lógicas del Grafo:

Para describir formalmente las restricciones estructurales y condiciones de la red, se definen las siguientes proposiciones lógicas:

1. **Grafo Conexo (Conectividad):** Para todo  $u, v$  perteneciente a  $V$  (si  $u$  es distinto de  $v$ , entonces existe  $P$  tal que  $P$  es un camino entre  $u$  y  $v$ ).
2. **Pesos Estrictamente Positivos:** Para toda arista  $(u, v)$  perteneciente a  $E$ , se cumple que el peso  $w(u, v)$  es mayor que 0.

El problema se define dado un nodo de origen " $s$ " perteneciente a  $V$  y un destino " $t$ " perteneciente a  $V$ , donde se busca un camino  $P = (s = v_0, v_1, v_2, \dots, v_k = t)$ . El costo total del camino  $C(P)$  está dado por la suma de sus pesos:  $C(P) = \text{suma desde } i=0 \text{ hasta } k-1 \text{ de } w(v_i, v_{i+1})$ .

La ruta óptima es el camino  $P^*$  que minimiza dicho costo entre todas las trayectorias posibles en el grafo conexo:  $P^* = \arg \min C(P)$ .

### Fuentes Verificables de Información:

Las conexiones viales reales y las distancias en kilómetros asignadas como pesos a cada arista del grafo fueron extraídas y validadas utilizando la plataforma oficial de Google Maps (herramienta de cálculo de rutas e itinerarios), para los mapas viales del Instituto Nacional de Vías (INVIAS) de Colombia y para planificar las rutas tramo por tramo y ver la distancia vial usamos la herramienta Rome2Rio la cual nos muestra la distancia de conducción calculada por carretera en tiempo real.

### 3. Metodología: Algoritmo de Camino Mínimo

Para la resolución de este problema, se seleccionó e implementó el Algoritmo de Dijkstra. La elección se fundamenta en las propiedades topológicas del grafo modelado: al representar distancias físicas entre ciudades, la función de peso "w" siempre devuelve valores estrictamente positivos ( $w > 0$ ). Dijkstra es el algoritmo idóneo, ya que garantiza encontrar el camino mínimo de fuente única en grafos sin aristas de peso negativo, operando mediante la relajación sucesiva de los nodos más cercanos.

Comparación con alternativa: Una alternativa matemática posible es el algoritmo de Bellman-Ford. Si bien Bellman-Ford cumple el mismo objetivo, está diseñado para tolerar grafos con pesos negativos. Esto conlleva una complejidad temporal superior de  $O(|V||E|)$ . Dado que la red de ciudades implementada carece de distancias negativas, el uso de Dijkstra apoyado por una cola de prioridad reduce drásticamente el costo computacional a  $O((|V| + |E|) \log |V|)$ , resultando significativamente más eficiente y pertinente para el contexto del proyecto.

### 4. Implementación

La solución se desarrolló en Python, siguiendo un paradigma modular que separa la lógica del grafo (cálculo), la optimización algorítmica y la visualización de datos. El sistema integra cuatro librerías clave que aseguran eficiencia y escalabilidad.

#### 4.1 Componentes Técnicos:

- **NetworkX:** Utilizada como motor para la estructura de datos abstracta. Permite definir la red interurbana como un grafo no dirigido (`nx.Graph()`), donde los nodos representan ciudades y las aristas almacenan la propiedad `weight` (distancia en km). Esta librería provee acceso de complejidad constante  $O(1)$  para la consulta de adyacencias.
- **heapq:** Esencial para la optimización del algoritmo de Dijkstra. La librería implementa internamente un Min-Heap (montículo binario) sobre una lista convencional. Esto garantiza que la operación de extracción del nodo con menor distancia acumulada se realice en tiempo logarítmico  $O(\log |V|)$ ,

optimizando drásticamente el rendimiento en comparación con una búsqueda lineal.

- **Tkinter:** Gestión de la Interfaz Gráfica de Usuario (GUI). Permite una interacción intuitiva mediante widgets `tkk.ComboBox` para la selección de ciudades origen/destino, asegurando la integridad de los datos mediante estados de solo lectura.
- **Matplotlib:** Renderizado gráfico del grafo. Se implementa en dos capas: una base para la topología total de la red y una capa superior que grafica exclusivamente el camino óptimo  $P^*$  con un grosor de trazo incrementado (`width=3.5`) y color diferenciado (`crimson`), facilitando la interpretación visual de la solución.

## 4.2 Estructura de Datos y Justificación del Algoritmo

La selección del algoritmo de Edsger Dijkstra como el motor lógico del programa se fundamenta en tres criterios teóricos estrictos:

1. **Naturaleza Homogénea y Positiva de los Pesos:** Al trabajar con distancias reales en kilómetros, se garantiza que  $w(u, v) \geq 0$ .
2. **Garantía de Optimalidad Absoluta:** Dijkstra asegura encontrar siempre la ruta terrestre más corta posible en grafos conexos con pesos no negativos.
3. **Eficiencia Asintótica:** Apoyado por una cola de prioridad mínima, reduce el costo computacional a  $O((|V| + |E|) \log |V|)$  frente a alternativas como Bellman-Ford.

Para la construcción computacional, las conexiones reales se almacenaron en una lista de tuplas que luego pueblan el objeto de `NetworkX`. El fragmento de código que inicializa la estructura es:

```
CONEXIONES = [  
    ("Bogotá", "Medellín", 415),  
    ("Bogotá", "Ibagué", 200),  
    ("Bogotá", "Villavicencio", 120),  
    ("Bogotá", "Bucaramanga", 400),  
    ("Medellín", "Cali", 420),  
    ("Medellín", "Manizales", 200),  
    ("Medellín", "Montería", 400),  
    ("Cali", "Ibagué", 270),  
    ("Cali", "Pasto", 380),  
    ("Manizales", "Pereira", 50),  
    ("Pereira", "Ibagué", 120),
```

```
( "Bucaramanga", "Cúcuta", 230),  
( "Bucaramanga", "Santa Marta", 540),  
( "Montería", "Sincedejo", 120),  
( "Sincedejo", "Cartagena", 170),  
( "Cartagena", "Barranquilla", 120),  
( "Barranquilla", "Santa Marta", 100),  
( "Bucaramanga", "Medellín", 390),  
( "Sincedejo", "Medellín", 460),  
( "Cúcuta", "Bogotá", 550),  
( "Pereira", "Cali", 210),  
( "Villavicencio", "Bucaramanga", 490)  
]
```

```
def crear_grafo():  
    G = nx.Graph()  
    G.add_weighted_edges_from(CONEXIONES)  
    return G
```

### 4.3 Implementación del Cálculo de la Ruta Óptima

El cálculo del camino mínimo se gestiona dentro de la función `calcular_ruta()`, invocando los métodos óptimos de la librería para resolver la búsqueda del parámetro  $P^* = \arg \min C(P)$ . El núcleo algorítmico implementado que procesa los datos de la interfaz es:

```
try:  
    ruta = nx.shortest_path(  
        G,  
        origen,  
        destino,  
        weight="weight"  
    )  
  
    distancia = nx.shortest_path_length(  
        G,  
        origen,  
        destino,  
        weight="weight"  
    )  
  
    texto = (
```

```

        f"Ruta óptima:\n"
        f"{' → '.join(ruta)}\n\n"
        f"Distancia: {distancia} km"
    )

    resultado.config(text=texto)

    dibujar_grafo(G, ruta)

except nx.NetworkXNoPath:
    resultado.config(
        text="No existe una ruta disponible."
    )

```

## 5. Resultados y Pruebas

A continuación, se presentan tres escenarios de prueba ejecutados exitosamente en la aplicación desarrollada:

| Prueba   | Origen    | Destino   | Ruta Óptima                                 | Costo (km) |
|----------|-----------|-----------|---------------------------------------------|------------|
| Prueba 1 | Bogotá    | Cali      | Bogotá → Ibagué → Cali                      | 470        |
| Prueba 2 | Cúcuta    | Manizales | Cúcuta → Bucaramanga → Medellín → Manizales | 820        |
| Prueba 3 | Cartagena | Bogotá    | Cartagena → Sincelejo → Medellín → Bogotá   | 1045       |

## 6. Conclusiones

El desarrollo del proyecto demostró empíricamente cómo los conceptos teóricos de matemática discreta son directamente aplicables a la resolución de problemas de logística e infraestructura. La dificultad técnica principal radicó en la sincronización del algoritmo matemático con la librería de visualización gráfica, evitando bloqueos en la interfaz mediante el correcto manejo de las figuras. Como mejora futura, el modelo matemático podría extenderse incorporando grafos dirigidos para simular sentidos vehiculares, o la integración de múltiples variables de peso (como tráfico en tiempo real) que dinamicen el cálculo del camino mínimo.

## 7. Repositorio en GitHub

El código fuente completo, los datos de configuración del grafo, el archivo requirements.txt y las instrucciones de ejecución (README.md) se encuentran alojados y disponibles públicamente en el siguiente enlace:

Enlace del repositorio:

<https://github.com/fmartinez2026-lab/proyecto-mate.discreta/blob/main/proyecto%20mate%20discreta.py>